Упражнение к PLF: прокат транспортных средств

> Класс: 5AKIF
> Подготовка к PLF (практической контрольной работе) в среду, 4 ноября 2026 года
> Время работы на PLF: 3 UE (урока)

Указания к этому упражнению

Это упражнение показывает, как выглядят задание и тестовая программа на PLF. PLF имеет такую же структуру: модель классов, описание классов и тестовая программа, которая начисляет баллы.

Скачайте файл *CarRentalService.7z* из упражнений к главе *Types*. Распакуйте файл и откройте файл *CarRentalService.sln*.

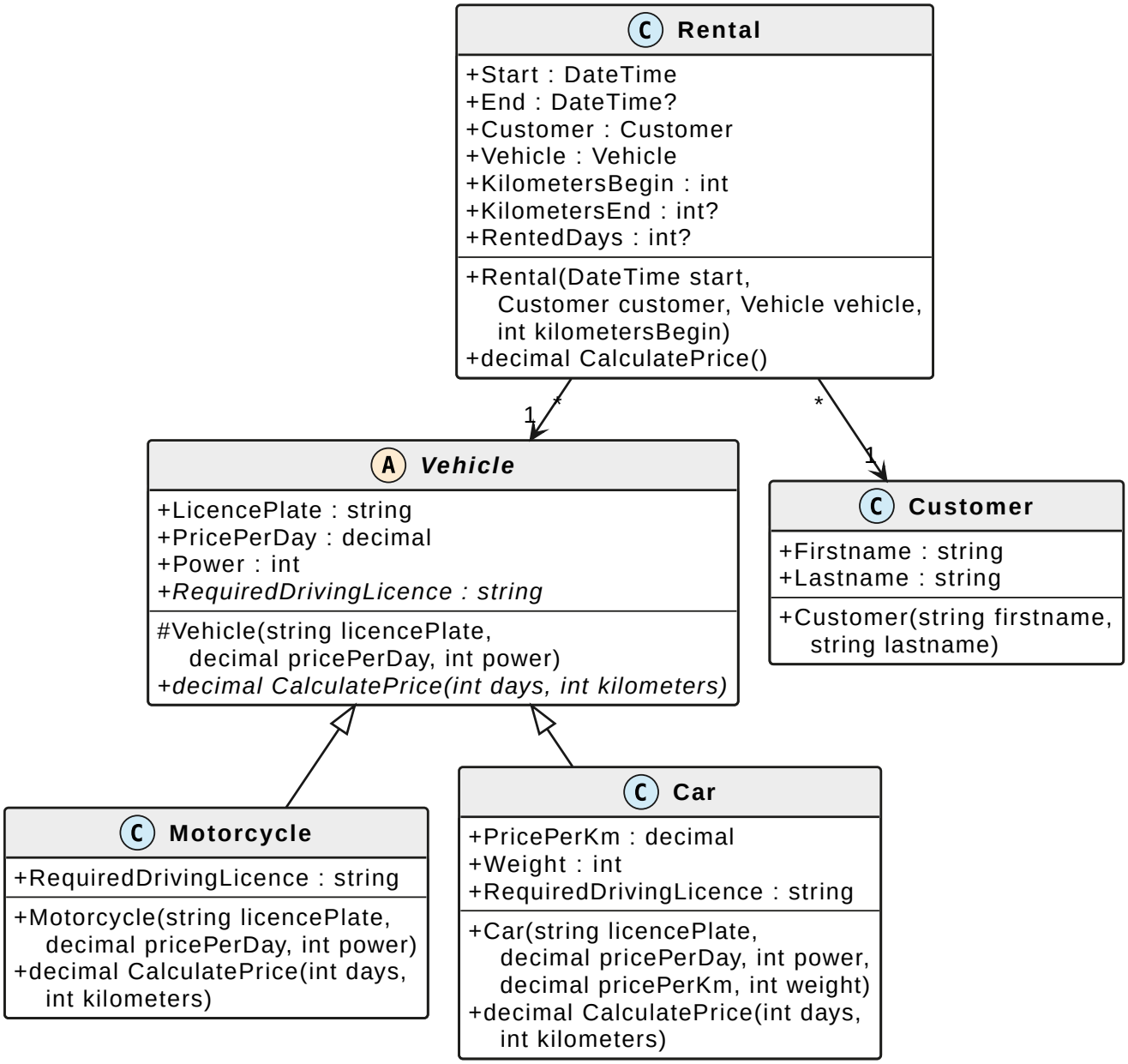


На PLF вы найдёте solution на диске *Z*. Там сначала впишите своё имя в файл *README.md*. В этом упражнении, наоборот, файл *README.md* содержит задание, и вписывать ничего не нужно.

Задача: прокат транспортных средств

Пункт проката сдаёт в аренду мотоциклы и автомобили. За мотоцикл вы платите цену за день. За автомобиль вы дополнительно платите за каждый пройденный километр. Для каждой аренды система сохраняет запись об аренде (*Rental*). Аренда остаётся открытой, пока транспортное средство не вернут.

Модель классов



Описание классов

Приватные поля, которые нужны для реализации, создайте сами.

Класс Vehicle

Абстрактный базовый класс для автомобилей и мотоциклов. Все properties доступны только для чтения (read-only).

- **LicencePlate** — номерной знак транспортного средства.
- **PricePerDay** — цена аренды за каждый начатый день.
- **Power** — мощность транспортного средства в кВт.
- Конструктор имеет модификатор *protected*.

- **RequiredDrivingLicence** — абстрактное property. Оно возвращает категорию водительского удостоверения, которая нужна для этого транспортного средства.
- **CalculatePrice()** — абстрактный метод. Он вычисляет цену по количеству дней аренды (*days*) и пройденным километрам (*kilometers*).

Класс Motorcycle

Все properties доступны только для чтения.

- **RequiredDrivingLicence** всегда возвращает *"A"*.
- **CalculatePrice()** вычисляет цену по формуле *days* × *PricePerDay*. Километры не учитываются.

Класс Car

Все properties доступны только для чтения.

- **PricePerKm** — цена за пройденный километр.
- **Weight** — масса в кг.
- **RequiredDrivingLicence** возвращает *"B"*, если автомобиль весит меньше 3500 кг. Иначе оно возвращает *"C"* (грузовик).
- **CalculatePrice()** вычисляет цену по формуле *days* × *PricePerDay* + *kilometers* × *PricePerKm*.

Класс Customer

Класс хранит только имя и фамилию клиента. Конструктор задаёт все properties. Все properties доступны только для чтения.

Класс Rental

Класс хранит одну аренду.

- **Start**, **Customer**, **Vehicle** и **KilometersBegin** задаются в конструкторе и доступны только для чтения.
- **Start** — начало аренды.
- **KilometersBegin** — показание счётчика километров (пробег) в начале аренды.
- **End** — конец аренды. Значение равно *null*, пока транспортное средство не вернули. При возврате значение задают извне класса.
- **KilometersEnd** — показание счётчика километров в конце аренды. Значение равно *null*, пока транспортное средство не вернули. При возврате значение задают извне класса.
- **RentedDays** — количество начатых дней, доступно только для чтения. Если *End* равно *null*, то и *RentedDays* равно *null*. Аренда, которая заканчивается в тот же день, длится 1 день.
- **CalculatePrice()** вычисляет цену аренды:

- Если *End* равно *null*, метод возвращает 0.
- Если *KilometersEnd* равно *null*, метод возвращает 0.
- Иначе метод возвращает результат *CalculatePrice()* арендованного транспортного средства. Передайте при этом *RentedDays* и пройденные километры (*KilometersEnd* – *KilometersBegin*).

Пример: Автомобиль с *PricePerDay* 10 евро и *PricePerKm* 5 евро арендуют с 10.11.2021 по 15.11.2021. Пробег в начале аренды — 100 км, в конце — 200 км.

- *RentedDays* равно 6 (10, 11, 12, 13, 14 и 15 ноября).
- *CalculatePrice()* возвращает 6 × 10 + 100 × 5 = 560 евро.



- Если вычесть одно значение *DateTime* из другого, получится *TimeSpan*. Количество начатых дней даёт такое выражение:

```
(int)(end - start).TotalDays + 1
```

- Это работает только с *DateTime*, но не с *DateTime?*. Поэтому сначала проверьте, равно ли *End* значению *null*. После этого значение можно получить через `End.Value`. То же самое относится к *KilometersEnd*.
- Литерал типа *decimal* записывается с *M* в конце, например `0.5M`.

Что нужно сделать

Создайте каждый класс в отдельном файле. Файлы *Program.cs* и *TestHelpers.cs* изменять **нельзя**.

Program.cs — тестовая программа. Она компилируется только тогда, когда существуют все классы со всеми properties и методами из модели. Обратите внимание и на имена параметров конструкторов, потому что тестовая программа их использует (например, `pricePerDay: 10`). Поэтому сначала создайте все классы. Методы и properties, которые вы ещё не реализовали, выбрасывают *NotImplementedException*. Затем запустите программу в проекте *CarRentalService.Application*. Для каждой проверки она показывает *OK* или *FAILED*. Если проверка выбрасывает исключение (exception), она считается не пройденной. Исключение вне проверки (например, при создании тестовых данных) завершает программу.

Полное решение показывает *OK* во всех 43 проверках. В конце программа показывает набранные баллы и оценку:

```
Result of the program in C:\...\CarRentalService.Application:
43 of 43 checks passed.
48 of 48 points. Grade: Sehr gut.
```

Оценивание

Чтобы получить положительную оценку, программа должна компилироваться. Программы, которые не компилируются или аварийно завершаются, получают оценку *Nicht genügend* (неудовлетворительно). В таблице перечислены все проверки тестовой программы. Тексты в таблице совпадают с выводом на экран. Каждая проверка с результатом *OK* приносит баллы из столбца *P*.

№	**Проверка (всего 48 баллов)**	**P**
Checking class implementation		
1	Rental does not have a default constructor.	1
2	Rental.Start is read-only.	1
3	Rental.Customer is read-only.	1
4	Rental.Vehicle is read-only.	1
5	Rental.KilometersBegin is read-only.	1
6	Rental.RentedDays is read-only.	1
7	Rental.End, KilometersEnd and RentedDays are nullable.	1
8	Customer does not have a default constructor.	1
9	Customer is immutable.	1
10	Vehicle does not have a default constructor.	1
11	Vehicle is abstract.	1
12	Vehicle has a protected constructor.	1
13	Vehicle is immutable.	1
14	Vehicle.PricePerDay is decimal and Vehicle.Power is int.	1
15	Vehicle.RequiredDrivingLicence is abstract.	1
16	Vehicle.CalculatePrice() is abstract.	1
17	Motorcycle does not have a default constructor.	1
18	Motorcycle is immutable.	1
19	Car does not have a default constructor.	1
20	Car is immutable.	1
21	Car.PricePerKm is decimal and Car.Weight is int.	1
Checking constructors		
22	Constructor of Customer sets the values.	1
23	Constructor of Motorcycle sets the values.	1
24	Constructor of Car sets the values.	1
25	Constructor of Rental sets the values.	1

№	Проверка (всего 48 баллов)	Р
Checking Motorcycle		
26	The required driving licence for motorcycles is A.	1
27	Motorcycle.CalculatePrice() returns days x PricePerDay.	2
Checking Car		
28	The required driving licence for cars < 3500 kg is B.	1
29	Car.CalculatePrice() returns days x PricePerDay + km x PricePerKm.	2
30	The required driving licence for cars >= 3500 kg is C.	1
Checking Rental		
31	The default value of Rental.KilometersEnd is null.	1
32	The default value of Rental.End is null.	1
33	Without an end date Rental.RentedDays is null.	1
34	Without End and KilometersEnd Rental.CalculatePrice() returns 0.	1
35	Rental.RentedDays returns the number of days started.	2
36	Without KilometersEnd Rental.CalculatePrice() returns 0.	1
37	Rental.CalculatePrice() uses the driven kilometers.	2
38	Without End Rental.CalculatePrice() returns 0.	1
39	Rental.RentedDays is 1 if the vehicle is returned on the same day.	1
40	Rental.RentedDays works across the end of a month.	1
41	Rental.RentedDays works across the end of a year.	1
42	Rental.CalculatePrice() charges the days if no kilometers were driven.	1
43	Rental.CalculatePrice() uses the price calculation of the rented vehicle.	2

48 - 42 балла: Sehr gut (1, отлично), 41 - 36 баллов: Gut (2, хорошо), 35 - 30 баллов: Befriedigend (3, удовлетворительно), 29 - 24 балла: Genügend (4, достаточно), 23 - 0 баллов: Nicht genügend (5, неудовлетворительно). Программа выводит название оценки на немецком языке.